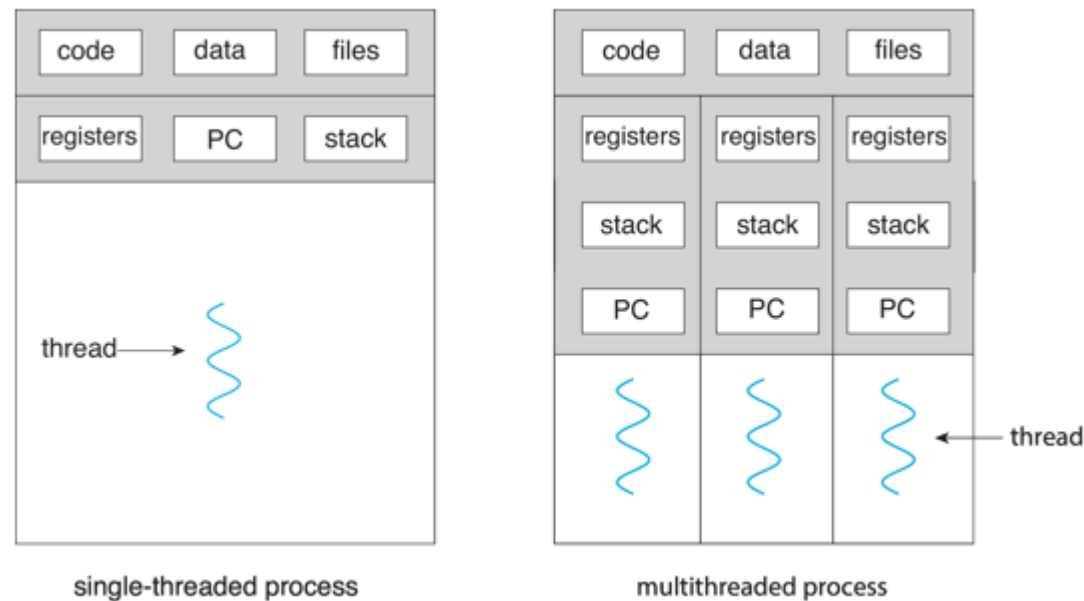


Chap4_Threads



Outline

1. Overview
2. Multicore Programming
3. Multithreading Models
4. Thread Libraries
5. Implicit Threading
6. Threading Issues
7. Operating System Examples
8. Parallelism

- In general, there are two types of parallelism: data parallelism and task parallelism. **Data parallelism** focuses on distributing subsets of the same data across multiple computing cores and performing the same operation on each core. Consider, for example, summing the contents of an array of size N . On a single-core system, one thread would simply sum the elements $[0] \dots [N - 1]$. On a dual-core system, however, thread A , running on core 0, could sum the elements $[0] \dots [N/2 - 1]$ while thread B , running on core 1, could sum the elements $[N/2] \dots [N - 1]$. The two threads would be running in parallel on separate computing cores.
Task parallelism involves distributing not data but tasks (threads) across multiple computing cores. Each thread is performing a unique operation. Different threads may be operating on the same data, or they may be operating

on different data. Consider again our example above. In contrast to that

- Amdahl's Law
- S is serial portion
- N processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times

As N approaches infinity, speedup approaches $1 / S$

Multithreading Models

- Many to one model
-

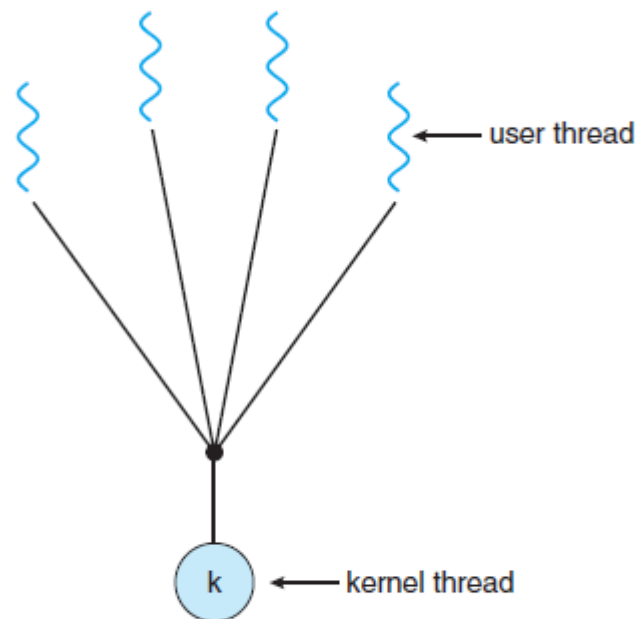


Figure 4.5 Many-to-one model.

- One to one model
-

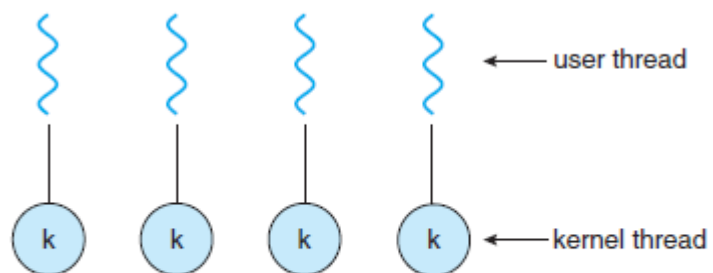


Figure 4.6 One-to-one model.

- drawback to this model is that creating a user thread requires creating the corresponding kernel thread. Because the overhead of creating kernel threads can burden the performance of an application, most implementations of this model restrict the number of threads supported by the system. Linux, along with the family of Windows operating systems, implement the one-to-one model

- The one to one model allows greater concurrency but there should not be too many threads

- **Many to many model**

-

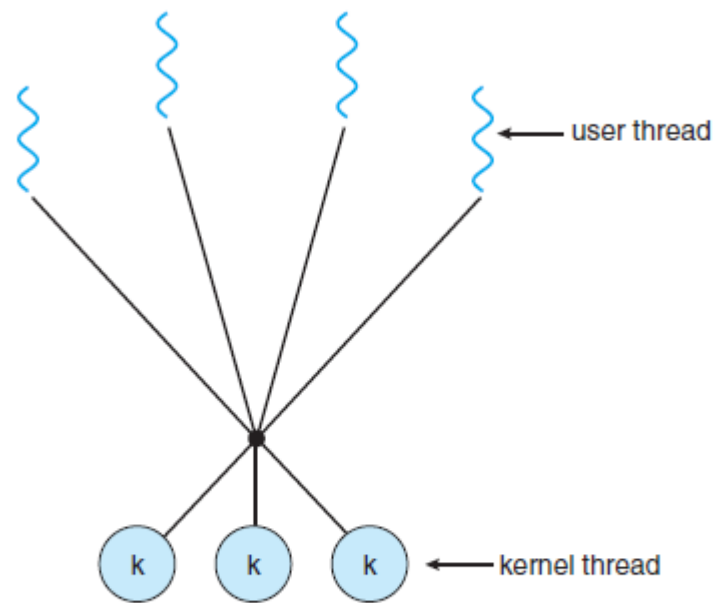


Figure 4.7 Many-to-many model.

- One variation on the many-to-many model still multiplexes many userlevel threads to a smaller or equal number of kernel threads but also allows a user-level thread to be bound to a kernel thread. This variation is sometimes referred to as **the two-level model** (Figure 4.8). The Solaris operating system supported the two-level model.

-

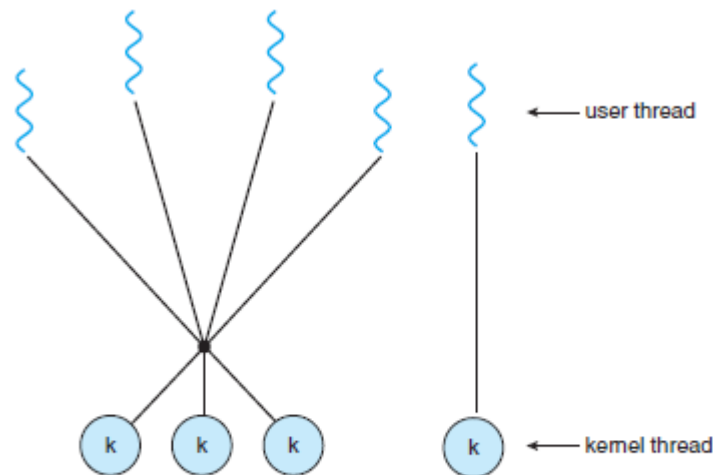


Figure 4.8 Two-level model.

- **Thread Libraries**

- 2 approaches
 1. User level library - has no kernel support
 2. Kernel level library - supported directly by OS
- 2 general strategies for thread creation
 - Asynchronous - parent resumes after creating child and executes concurrently

- Synchronous - Parent must wait for its children to finish execution. **Fork Join strategy**
 - 3 main libraries in use
 - **Pthreads**
 - **Windows threads**
 - **Java threads**
 - **Pthread**
 - Thread extension of POSIX standard,
 - Can be user level or kernel level
 - **Specification, not implementation**
 - API specifies behavior of the thread library, implementation is up to development of the library
 - Common in UNIX operating systems (Linux & Mac OS X)
 - **Java threads**
 - Java threads are managed by the JVM
 - Typically implemented using the threads model provided by underlying OS
 - Java threads may be created by:
 - Extending Thread class
 - Implementing the Runnable interface
 - ```
public interface Runnable
{
 public abstract void run();
}
```
    - Standard practice is to implement Runnable interface
    - ```
class Task implements Runnable
{
    public void run() {
        System.out.println("I am a thread.");
    }
}
```
- ```
Thread worker = new Thread(new Task());
worker.start();

try {
 worker.join();
}
catch (InterruptedException ie) { }
```
- A thing about JVM and host OS

## *THE JVM AND THE HOST OPERATING SYSTEM*

The JVM is typically implemented on top of a host operating system (see

Figure 16.10). This setup allows the JVM to hide the implementation details of the underlying operating system and to provide a consistent, abstract environment that allows Java programs to operate on any platform that supports a JVM. The specification for the JVM does not indicate how Java threads are to be mapped to the underlying operating system, instead leaving that decision to the particular implementation of the JVM. For example, the Windows XP operating system uses the one-to-one model; therefore, each Java thread for a JVM running on such a system maps to a kernel thread. On operating systems that use the many-to-many model (such as Tru64 UNIX), a Java thread is mapped according to the many-to-many model. Solaris initially implemented the JVM using the many-to-one model (the green threads library, mentioned earlier). Later releases of the JVM were implemented using the many-to-many model. Beginning with Solaris 9, Java threads were mapped using the one-to-one model. In addition, there may be a relationship between the Java thread library and the thread library on the host operating system. For example, implementations of a JVM for the Windows family of operating systems might use the Windows API when creating Java threads; Linux, Solaris, and Mac OS X systems might use the Pthreads API.

- **Implicit Threading**

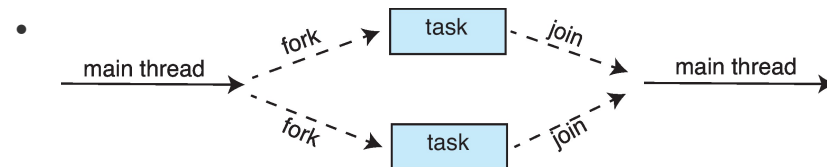
- Creation and management of threads done by compilers and run time libraries rather than programmers

- 6 methods

- Thread pools

- A fixed number of empty threads, at process startup are waiting to be assigned a task to execute
- This approach leads to less overhead of thread creation rather than just assigning a task to one already existing
- #threads can be based on factors such as : #CPUs, RAM, #concurrent client requests.

- Fork-Join



- Task(problem)

```

if problem is small enough
 solve the problem directly
else
 subtask1 = fork(new Task(subset of problem))
 subtask2 = fork(new Task(subset of problem))

 result1 = join(subtask1)
 result2 = join(subtask2)

 return combined results

```

- ```

ForkJoinPool pool = new ForkJoinPool();
// array contains the integers to be summed
int[] array = new int[SIZE];
    
```

```
int[] array = new int[SIZE];

SumTask task = new SumTask(0, SIZE - 1, array);
int sum = pool.invoke(task);
```

▪ OpenMP

- an API for C/C++/FORTRAN codes.
- identifies parallel regions as blocks of code that may run in parallel.
- creates as many threads as #CPU cores in the system.

- ```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
 /* sequential code */

 #pragma omp parallel
 {
 printf("I am a parallel region.");
 }

 /* sequential code */

 return 0;
}
```
- ```
#pragma omp parallel for
for (i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

OpenMP divides the work contained in the `for` loop among the threads it has created in response to the directive

```
#pragma omp parallel for
```

▪ Grand Central Dispatch

- Used by Apple's Mac and iOS
- A combo of C language extended, an API and a runtime library.
- Identifies extension of C and C++ languages as **Blocks**, a self contained unit of work.
- Schedules execution for blocks by placing them on a **dispatch queue** of two types - **serial and concurrent**.
 - serial – **blocks removed in FIFO order**, queue is per process, called main queue. Programmers can create additional serial queues within program
 - concurrent – removed in FIFO order but several may be removed at a time
- A simple example of a block is shown below:
 - `^ { printf("I am a block"); }`
 - ^ a caret inserted in front of the a pair of braces{ }.
- Also uses **thread pools**, allowing **#threads to grow and shrink**

▪ Inter threading building blocks

- Template library for designing parallel C++ programs
- A serial version of a simple for loop
 - ```
for (int i = 0; i < n; i++) {
 apply(v[i]);
}
```
- The same for loop written using TBB with `parallel_for` statement:



```

 ◦ parallel_for (size_t(0), n, [=](size_t i) {apply(v[i]);});

▪ Java API java.util.concurrent.

•
import java.util.concurrent.*;

public class ThreadPoolExample
{
public static void main(String[] args) {
 int numTasks = Integer.parseInt(args[0].trim());

 /* Create the thread pool */
 ExecutorService pool = Executors.newCachedThreadPool();

 /* Run each task using a thread in the pool */
 for (int i = 0; i < numTasks; i++)
 pool.execute(new Task());

 /* Shut down the pool once all threads have completed */
 pool.shutdown();
}

```

• **Signal Handling**

- Signals are used in UNIX systems to notify a process that a particular event has occurred.
- A signal handler is used to process signals
  - Signal is generated by particular event
  - Signal is delivered to a process
  - Signal is handled by one of two signal handlers:
    - default
    - user-defined
- Every signal has default handler that kernel runs when handling signal
  - User-defined signal handler can override default
  - For single-threaded, signal delivered to process

• **Thread cancellation**

- A thread that is to be canceled is often referred to as the target thread. Cancellation of a target thread may occur in two different scenarios:
  - 1. Asynchronous cancellation. One thread immediately terminates the target thread.
  - 2. Deferred cancellation. The target thread periodically checks whether it should terminate, allowing it an opportunity to terminate itself in an orderly fashion
- may set its cancellation state and type using an API.

| Mode         | State    | Type         |
|--------------|----------|--------------|
| Off          | Disabled | –            |
| Deferred     | Enabled  | Deferred     |
| Asynchronous | Enabled  | Asynchronous |

As the table illustrates, Pthreads allows threads to disable or enable

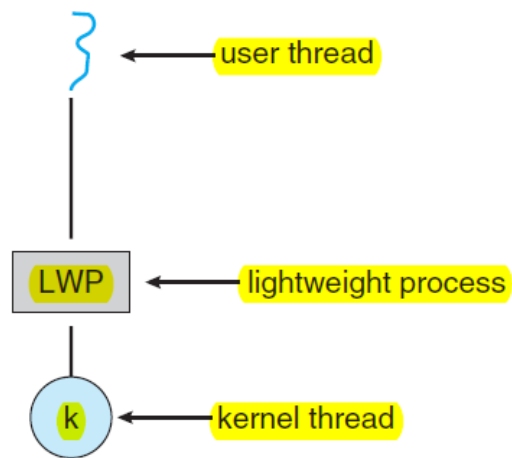
• **Thread local storage**

It is easy to confuse TLS with local variables. However, local variables are visible only during a single function invocation, whereas TLS data are

visible across function invocations. In some ways, TLS is similar to static data. The difference is that TLS data are unique to each thread. Most thread libraries—including Windows and Pthreads—provide some form of support for thread-local storage; Java provides support as well.

• **Scheduler activation**

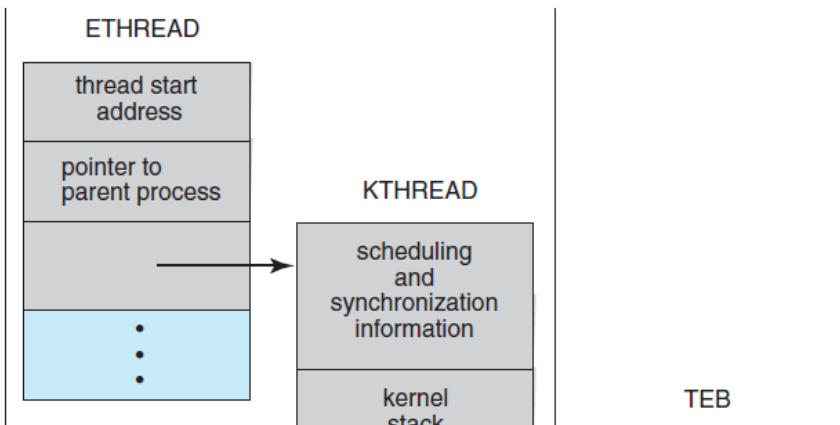
- communication between the kernel and the thread library, which may be required by the many-to-many and two-level models discussed
- 



**Figure 4.13** Lightweight process (LWP).

• **Windows threads**

- The general components of a thread include:
  - A thread ID uniquely identifying the thread
  - A register set representing the status of the processor
  - A user stack, employed when the thread is running in user mode, and a kernel stack, employed when the thread is running in kernel mode
  - private storage area used by various run-time libraries and dynamic link libraries (DLLs)
- The register set, stacks, and private storage area are known as the context of
- the thread.
- The primary data structures of a thread include:
  - • ETHREAD—executive thread block
  - • KTHREAD—kernel thread block
  - • TEB—thread environment block





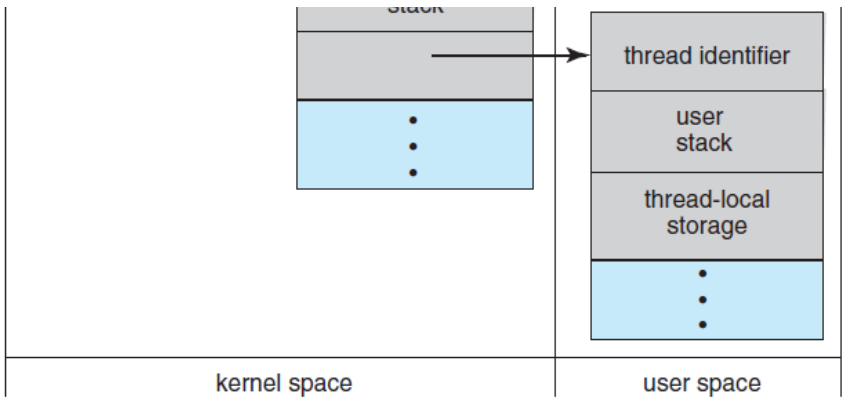


Figure 4.14 Data structures of a Windows thread.

• **LINUX**

Linux also provides the ability

to create threads using the clone() system call. However, Linux does not distinguish between processes and threads.

Using clone() in this fashion is equivalent to creating a thread as described in this chapter, since the parent task shares most of its resources with its child task. However, if none of these flags is set when clone() is invoked, no sharing takes place, resulting in functionality similar to that provided by the fork() system call

- 

| flag          | meaning                            |
|---------------|------------------------------------|
| CLONE_FS      | File-system information is shared. |
| CLONE_VM      | The same memory space is shared.   |
| CLONE_SIGHAND | Signal handlers are shared.        |
| CLONE_FILES   | The set of open files is shared.   |

Figure 4.15 Some of the flags passed when clone() is invoked.